Part 8 — Troubleshooting Scenarios

These three answers are written. I did not break the cluster to produce them. The commands are the ones I would run on our hadoop namespace, and the numbers I use for heartbeats, replication, and queues are the ones already set on this lab.

1. NameNode stays in safe mode after a restart

If the NameNode on our cluster came up in safe mode after a restart and stayed there, I would not run dfsadmin -safemode leave as the first command. Safe mode means the NameNode is up and it has the namespace, but it is refusing writes until enough DataNodes have reported enough block replicas. On a restart that is normal for a short time. It becomes a problem only when the NameNode log stops counting down and the same gap remains.

I would start from the VM, in the hadoop namespace, and read the current state before I change anything.

```
hdfs dfsadmin -safemode get
hdfs dfsadmin -report
hdfs dfsadmin -printTopology
kubectl get pods -n hadoop -l app=datanode -o wide
kubectl logs -n hadoop namenode-0 --tail=200
```

I run those through kubectl exec on namenode-0, the same way I did for fsck in Part 2. safemode get should print Safe mode is ON. The report tells me how many DataNodes are live, how many are dead, and whether missing blocks are still moving. printTopology tells me whether the live nodes are the ones I expect or whether one of them registered under a name the NameNode cannot use. The pod list tells me whether Kubernetes has actually started the DataNodes. The NameNode log is the piece that says why safe mode is holding. The line I look for is the one that says how many blocks have been reported, how many more it still needs to reach the threshold, and whether it is only in the safe mode extension.

On Hadoop 3.3 the usual settings are dfs.namenode.safemode.threshold-pct at 0.999, dfs.namenode.safemode.extension at 30000 milliseconds, dfs.namenode.safemode.min.datanodes at 0, and dfs.namenode.replication.min at 1. I would print those with hdfs getconf -confKey so I know which number the NameNode is waiting on. The threshold is a fraction of blocks that have at least the minimum replication. The extension is only an extra 30 seconds after that fraction is already met. If the log says the extension is counting down, I wait. If the log says it still needs additional blocks and that number is not falling, waiting alone will not finish the job.

On our cluster the next check is the DataNode pods, because we already hit a startup failure that looks like this. When datanode-2 was added, the new volume was owned by root and the DataNode process runs as uid 1000. It crashed in a loop, so it never sent a block report. I would look for CrashLoopBackOff, then read that pod log for DiskErrorException or NativeIO chmod EPERM. The fix we already use is the init container that runs chown 1000:100 on the data directory before the DataNode starts. Until that pod is 1/1 Running and shows up under Live datanodes, I leave safe mode alone.

I would also compare the live count with what we configured. We run three DataNodes. If the report shows one or two, I stay in the wait path. Our heartbeat timeout is about 10 minutes 30 seconds, from dfs.namenode.heartbeat.recheck-interval 300000 and dfs.heartbeat.interval 3. After a restart the DataNodes

usually report much sooner than that, but a slow block report on a large volume can take several minutes. I rerun dfsadmin -report every minute. If Live datanodes is climbing and the "blocks still needed" number in the NameNode log is dropping, the restart is still healthy and I wait.

If all three DataNodes are live, each one has sent a block report, and the NameNode still wants more blocks, then some blocks have no replica anywhere. I find them with:

```
hdfs fsck / -files -blocks -locations
hdfs fsck / -list-corruptfileblocks
```

fsck will fail to finish cleanly while safe mode is on for some checks, so I read the NameNode log and the fsck output together. For a file we still need, I restore it from a snapshot or from the copy we kept outside HDFS. We already created a snapshot under /datasets/.snapshot, so that is the first place I would look for the taxi data. For a file we can afford to lose, I delete that file once the NameNode will accept the delete, or I move it aside after leaving safe mode. Deleting the bad files is what lets the reported-block percentage reach the threshold. Lowering threshold-pct in hdfs-site.xml would also make safe mode exit, and I would treat that as hiding the missing data. I would not change it just to get a green prompt.

The decision is based on whether another block report can still arrive. I wait when any expected DataNode is not live yet, when the pods are still starting, when the log shows the extension timer, or when the count of blocks still needed is going down. I force the leave only when every DataNode we expect is live, the block reports have finished, the remaining missing blocks are a known set of files, and I have either restored those files or accepted that they are gone. The command is hdfs dfsadmin -safemode leave. That command does not rebuild the missing replicas. It only lets clients write again. If I run it while a DataNode is still starting, the NameNode can mark blocks missing that would have been reported a minute later, and clients start failing reads that would have worked if I had waited.

After a forced leave I run dfsadmin -report and fsck again, confirm missing blocks is zero or matches the short list I already accepted, and I watch Under replicated blocks until it returns to the normal level. I also check the NameNode UI on port 9870, through a port-forward, and confirm the safe mode banner is gone. If safe mode comes back by itself, a DataNode has died again and I go back to the pod logs instead of leaving safe mode a second time.

2. Spark Structured Streaming lag while batches queue

If a Spark Structured Streaming job is reading Kafka and the consumer lag keeps growing, while the Spark UI shows tasks finishing and new batches sitting in a queue, I would not start by adding executors. The cluster having free memory and free cores only means YARN has room. It does not mean this job is able to use that room, and it does not mean a micro-batch is finishing faster than the next one arrives. Structured Streaming runs one micro-batch at a time. If a batch takes longer than the trigger interval, the next ones wait on the driver. The tasks you see completing are the tasks of the batch that is already running. The queued batches are the ones that have not been allowed to start, and Kafka lag grows because the offsets move forward only when a batch commits.

I would open the Streaming tab on the driver UI first. For a job on our cluster that is the Spark UI of the driver pod, reached with kubectl port-forward. The two numbers that split the problem are processing time and scheduling delay. Processing time is how long the batch worked once it started. Scheduling delay is how long it

sat in the queue before it started. I also read input rows per second against process rows per second. Then I check the same consumer group in Kafka so I can see whether the lag is on every partition or only on one.

```
kubectl exec -n hadoop kafka-0 -- kafka-consumer-groups.sh --bootstrap-server kafka-0.kafka-hs.hadoop.svc.cluster.local:9092 --describe --group <group>
```

On the Spark side I look at the stage summary for that batch: number of tasks, duration of the slowest task, and shuffle read. On YARN I look at the application and at the queue, because our capacity scheduler gives production 60 percent with a max of 80 percent and development 40 percent with a max of 100 percent. Free memory in the cluster can sit in the other queue while this job is capped.

The first cause is a trigger that is shorter than the batch. If processing time is 8 seconds and the trigger is 1 second, scheduling delay climbs in a straight line and the Kafka group lag climbs with it. The tasks are successful. They are just too slow for the rate. I would either raise the trigger interval to something above the real batch time, or set maxOffsetsPerTrigger on the Kafka source so each batch reads a bounded number of records. After a pause, an unbounded batch tries to swallow the whole backlog in one go, and that single batch makes the queue worse. I would re-run and watch the process rate until it stays above the input rate.

The second cause is parallelism. The Kafka source creates one task per topic partition unless minPartitions is set higher. A topic with 6 partitions, which is what we plan for app-events, gives the batch 6 tasks. The rest of the YARN cores sit idle, so the cluster looks free while the batch is still slow. I confirm this when the stage has a handful of tasks and each task's executor CPU time is high. The fix is more Kafka partitions, or minPartitions on the read so Spark splits a partition into more tasks, and then a matching spark.sql.shuffle.partitions for the later stages. Adding executors without adding partitions does not shorten that stage.

The third cause is skew, and it matches the UI description closely. Most tasks complete, one task keeps running, and the batch cannot commit until that task finishes, so new batches queue. kafka-consumer-groups shows the lag on one partition, and the stage task summary shows one task with far more records. I would look at the key that dominates that partition. For a streaming aggregation the fix is to spread the hot key, with salting on the key or a repartition that breaks the skew, and a watermark so state for late events does not grow forever. AQE can help the shuffle stages. It does not fix a Kafka partition that is already hot before the shuffle.

The fourth cause is the end of the batch rather than the tasks. The UI shows every task succeeded, then a long quiet gap before the batch is marked completed, and the queue grows during that gap. That is usually the output commit or the checkpoint. On HDFS the v1 file output committer does the final rename work on the driver after the tasks are done. I would set the committer to algorithm version 2 for this job and point the checkpoint at a directory on HDFS with enough room. If the checkpoint directory is huge or the NameNode is busy, every batch pays that cost even when the executors were fast. Stateful operations make this worse. A stream-stream join or a flatMapGroupsWithState with no watermark keeps every key. The State Store rows in the UI grow, RocksDB compaction stalls a task, and the batch time creeps up over hours. The fix is a watermark that expires old state, and a checkpoint that we can afford to rewrite.

The fifth cause is resources that the UI says are free because the job let them go. With dynamic allocation, executors can exit between batches. The next batch then sits in scheduling delay while YARN allocates new ones, and the cluster monitor shows idle CPU the whole time. I would set a small minExecutors so at least a few executors stay, and raise the executor idle timeout so they are not removed between short batches. I would also check the queue. If this job was submitted to development, production can be using its share and the app will queue for containers even though a node looks empty. The RM UI queue page shows the allocated and pending

memory for that queue. Moving the stream to the queue that is supposed to own it, or lowering its executor memory so the ask fits the queue, removes that wait.

The sixth cause is the Kafka consumer being dropped. If one batch runs longer than max.poll.interval.ms, the broker decides the consumer is dead, the group rebalances, and the lag jumps even though Spark tasks were completing. The consumer group flips state, and the driver log shows a rebalance or a lost offset. I would bound the batch with maxOffsetsPerTrigger so a batch cannot run that long, and set the Kafka option so the poll interval is above the worst batch we are willing to allow.

I would change one of these at a time and watch the Streaming tab for ten minutes. The fix worked when processing time stays under the trigger, scheduling delay goes back down instead of climbing, and the Kafka lag for every partition stops growing.

3. Rolling upgrade from Hadoop 3.3.4 to 3.3.6 on Kubernetes

This is the runbook I would follow to move the Hadoop image from 3.3.4 to 3.3.6 on our Kubernetes cluster without taking HDFS or YARN down for the whole upgrade. I would not change HBase, Kafka, or Solr in the same change. They keep running as clients of HDFS. I would also not combine this with a config rewrite. The only intended change in each step is the image tag, from apache/hadoop:3.3.4 to apache/hadoop:3.3.6.

Our lab today runs one NameNode pod. Restarting that one pod is a metadata pause, so I would not call the lab layout zero downtime. Before the upgrade window I would add the pieces a rolling metadata upgrade needs: a standby NameNode and three JournalNodes, with the active NameNode already writing edits to the journal quorum. The DataNode roll can be zero downtime on the current layout as long as each pod comes back on its own PVC. The NameNode roll is zero downtime only after the standby exists. The steps below are the HA procedure. On the lab, I would do the DataNode and NodeManager rolls the same way, and I would treat the single NameNode restart as the last, shortest step.

Pre-checks, all of them before I change an image. I run these from namenode-0 with kubectl exec.

```
hdfs dfsadmin -safemode get
hdfs fsck /
hdfs dfsadmin -report
hdfs dfsadmin -rollingUpgrade query
yarn node -list
```

Safe mode must be off. fsck must be healthy, with no missing blocks and no corrupt blocks. Every DataNode and every NodeManager we expect must be live. rollingUpgrade query must say that no upgrade is already in progress. I take a copy of the NameNode metadata with hdfs dfsadmin -fetchImage into a directory that is not inside the pod's writable layer, and I confirm the DataNode PVCs are bound. I pull apache/hadoop:3.3.6 on the VM and load it into Kind before the window, so a node does not sit in ImagePullBackOff in the middle of the roll. I write down the current image on each StatefulSet so the rollback command is a tag I have already seen running. I also check disk headroom. With replication factor 3 and exactly three DataNodes, stopping one DataNode leaves the third replica with nowhere else to go. That is acceptable only because the same PVC comes back with the same blocks, which is what we saw when datanode-2 returned in Part 2. I would not delete a PVC, and I would not decommission a node, during this upgrade.

I put the NameNode into rolling-upgrade prepare next.

```
hdfs dfsadmin -rollingUpgrade prepare
hdfs dfsadmin -rollingUpgrade query
```

Prepare writes a checkpoint and records that a rollback is still possible. I do not finalize until the end. Finalize is what closes the door on rolling back.

Upgrade order. JournalNodes first, one pod at a time, if we added them for HA. Then the standby NameNode. Then the DataNodes, one pod at a time. Then I fail over to the upgraded standby and upgrade the NameNode that used to be active. Then the NodeManagers, one pod at a time. Then the standby ResourceManager, a failover, and the other ResourceManager. The JobHistory pod last. Clients and the gateway image last of all. I never have two DataNodes down together on this cluster, because we only have three and the factor is 3.

For a DataNode I use the StatefulSet partition so Kubernetes updates only the highest ordinal.

```
kubectl patch statefulset datanode -n hadoop --type merge -p
"{"spec":{"updateStrategy":{"type":"RollingUpdate","rollingUpdate":{"partition":2}}}}"
kubectl set image statefulset/datanode datanode=apache/hadoop:3.3.6 -n hadoop
```

I wait until datanode-2 is 1/1 Running. I then check dfsadmin -report for that host as live, and I check that it registered with the same storage id as before, which confirms it mounted the old PVC and did not come up as an empty node. I run fsck on /datasets/nyc-taxi and I wait until under-replicated blocks for that path is back to 0 before I touch the next pod. On our settings a DataNode that stays gone for about 10 minutes 30 seconds is marked dead. The goal is to have the pod back well inside that window, the way a normal restart returns, so the NameNode never drops its replicas. Then I set partition to 1, wait for datanode-1 the same way, then partition to 0 for datanode-0.

I upgrade NodeManagers with the same partition pattern on the nodemanager StatefulSet. After each NodeManager I run yarn node -list and I only continue when that node is RUNNING again. A NodeManager on 3.3.6 can rejoin a ResourceManager that is still on 3.3.4 for this small jump inside 3.3, which is why the NodeManagers go before the ResourceManager. After the NodeManagers are all on 3.3.6 I upgrade the ResourceManager. With RM HA I upgrade the standby, fail over, then upgrade the other one. With our single ResourceManager I restart that one pod only after the NodeManagers are healthy, and I expect a short pause in new allocations while it starts.

Validation before I finalize. safemode get is off. dfsadmin -report shows the same live DataNode count as the pre-check, on the 3.3.6 image. fsck / is healthy and the taxi path still has average replication 3. yarn node -list shows every NodeManager. I submit the same WordCount we already ran, on the production queue, and I wait for SUCCEEDED. I run hbase hbck style status only far enough to see the master still sees the region servers, and I run a Solr query we already know, so I know the HDFS clients survived the roll. Then:

```
hdfs dfsadmin -rollingUpgrade finalize
hdfs dfsadmin -rollingUpgrade query
```

Query after finalize should show that no rollback is in progress. I keep the 3.3.4 image loaded on the Kind nodes until the next day in case an application problem shows up that fsck did not catch.

Rollback, and this only exists if I have not finalized. I set the image back to apache/hadoop:3.3.4 in the reverse order: ResourceManager, NodeManagers, NameNodes, DataNodes, JournalNodes. I start the NameNode with the rolling upgrade rollback if query still says a rollback is required, and I run hdfs dfsadmin -rollingUpgrade rollback. I validate with the same safemode, report, fsck, and WordCount checks. If I had already finalized, that rollback command is no longer valid. The way back is the fsimage I saved with fetchImage, plus the DataNode

PVCs if they were snapshotted before the window. That is why finalize is a separate step at the end and not something I run as soon as the first DataNode looks healthy.